INSTITUTO

TECNOLOGICO

DE REYNOSA

PRÁCTICA GRUPAL

UNIDADES 1 Y 2

PROGRAMAS CON INTERFAZ GRÁFICA Y ACCESO A BASE DE DATOS

EQUIPO 4

NOMBRE DE LOS ESTUDIANTES:

CAMBRÓN ESCOBAR GUILLERMO (24580005)
GARCIA CONTRERAS GENESIS NERINA (22580080)
CONTRERAS JUAREZ EZRY ALEXANDRA (22580067)
ISIDRO SANCHEZ LISSET (21580044)
RAMIREZ CHAVARRIA ERIKA (21580059)

CARRERA: ING. TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIONES

ASIGNATURA: PROGRAMACIÓN 2

DOCENTE: MIRIAM PUENTE JIMENEZ

REYNOSA, TAMPS. 15/ABR/2026

No.	Matricula	Nombre	Programa asignado	Responsabilidades adicionales
1	24580005	Guillermo Cambron Escobar	Programa #9, Consumo de agua	Lider de equipo: Login
2	21580044	Lisset Isidro Sanchez	Programa # 7 Hectareas de algodon y maiz	Integracion de BD
3	22580067	Ezry Alexandra Contreras	Programa # 3 Casa de cambio	Menu principal / navegacion
4	22580080	Genesis Nerina Garcia Contreras	Programa # 11 Lavanderia	Pruebas / validacion
5	21580059	Erika Ramirez Chavarria	Programa # 24 Pago empleados	Documentacion / manual

Liga del repositorio

https://github.com/LissetSanchez25/Planilla

Requisitos del sistema

Sistemas operativos (OS)

Requisito mínimo: Windows 10

Requisito recomendado: Windows 11

Memoria RAM

Requisito mínimo: 4 GB

Recomendado: 8 GB o más

JDK

Se requiere la instalación del Java Development Kit (JDK) versión 17 o superior para la correcta compilación y ejecución del sistema.

.NET FRAMEWORK/NET

Se requiere la instalación de .NET 6.0 o superior y Visual Studio para la correcta compilación y ejecución de las aplicaciones desarrolladas en C#.

SGBD

Se requiere la instalación de un Sistema Gestor de Base de Datos (SGBD), específicamente MySQL Server versión 8.0 o superior, para el almacenamiento y gestión de la información del sistema. Asimismo, se recomienda el uso de herramientas de administración como MySQL Workbench versión 8.0 Community Edition para la correcta manipulación, consulta y mantenimiento de la base de datos.

Pasos para instalar la base de datos y configurar la cadena de conexión.

Instalación de la Base de Datos y Configuración de la Conexión (Java + MySQL)

1. Instalación de MySQL Server

1. Descargar el instalador oficial desde el sitio de MySQL.
2. Ejecutar el archivo **MySQL Installer**.
3. Seleccionar el tipo de instalación:
    - *Developer Default* (recomendado)
    - o *Custom* si se desea seleccionar componentes manualmente.
4. Instalar **MySQL Server 8.0**.
5. Configurar el servidor:
    - Tipo: *Standalone MySQL Server*
    - Configuración: *Development Computer*
6. Definir credenciales:

    - Usuario: root
    - Contraseña: (ejemplo: 1234)
7. Configurar puerto:
    - Puerto por defecto: **3306**
8. Finalizar instalación y verificar que el servicio esté activo.

2. Instalación de MySQL Workbench

1. Dentro del mismo instalador, seleccionar e instalar MySQL Workbench.
2. Abrir MySQL Workbench.
3. Crear una nueva conexión:
    - Hostname: localhost
    - Puerto: 3306
    - Usuario: root
4. Probar la conexión (*Test Connection*).
5. Guardar la conexión.

3.Creación de la Base de Datos

1. Abrir MySQL Workbench.
2. Conectarse al servidor.
3. Crear la Base de datos.
4. Verificar que la base de datos y tablas se hayan creado correctamente.

5. Descarga del Driver JDBC

6.

1. Descargar el conector JDBC desde el sitio oficial:
    - MySQL Connector/J
2. Descargar el archivo .zip.
3. Extraer el contenido.
4. Localizar el archivo .jar (ejemplo: mysql-connector-j-8.x.x.jar).

5. Agregar el Driver en NetBeans

1. Abrir Apache NetBeans.
2. Ir al proyecto Java.
3. Dar clic derecho en el proyecto → **Properties**.
4. Seleccionar **Libraries**.
5. Hacer clic en **Add JAR/Folder**.
6. Seleccionar el archivo .jar del driver descargado.
7. Confirmar y guardar cambios.

8. Configuración de la cadena de conexión

Utilizar la siguiente cadena de conexión:

```
Connection con = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/planilla",
"root",
"1234"
);
```

9. Carga del Driver en Java

Antes de la conexión, asegurarse de cargar el driver:

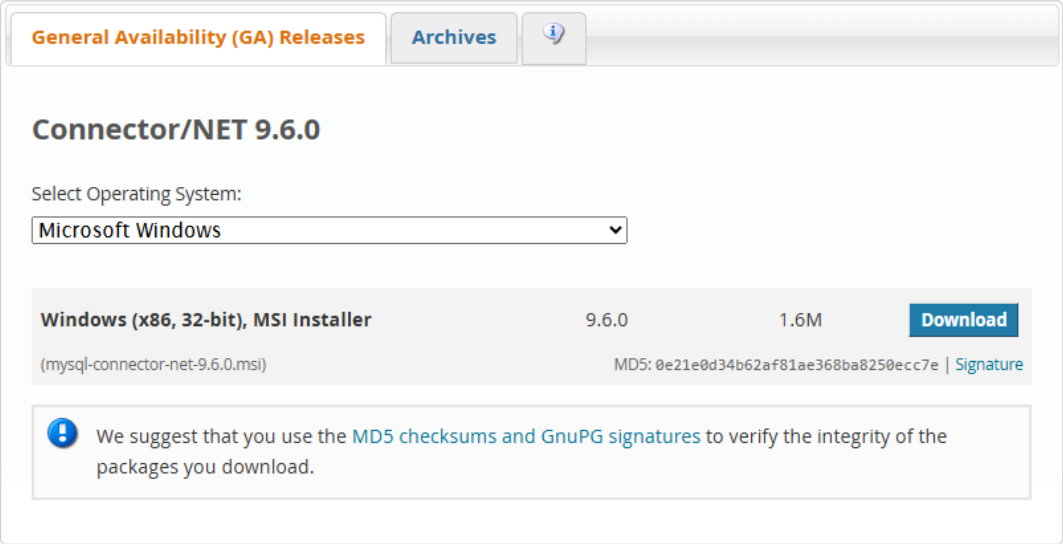
```
Class.forName("com.mysql.cj.jdbc.Driver");
```

10. Prueba de Conexión

1. Ejecutar el programa Java.
2. Verificar que no existan errores de conexión.
3. Validar acceso a la base de datos mediante consultas:

11. DRIVER PARA C#

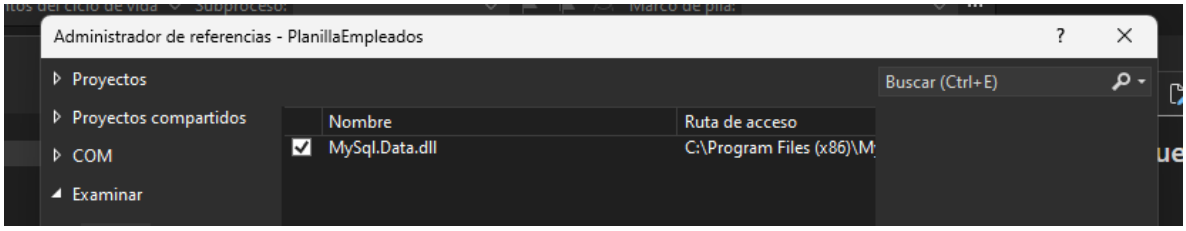
1. Descargar el conector NET desde el sitio oficial:



2. Descargar el archivo .zip.
3. Extraer el contenido.
4. Localizar el archivo

12. Agregar el Driver en Visual Studio

1. Abrir Visual studio
2. Ir al proyecto
3. Abrir el Administrador de referencias y agregarlo
4. Aceptar



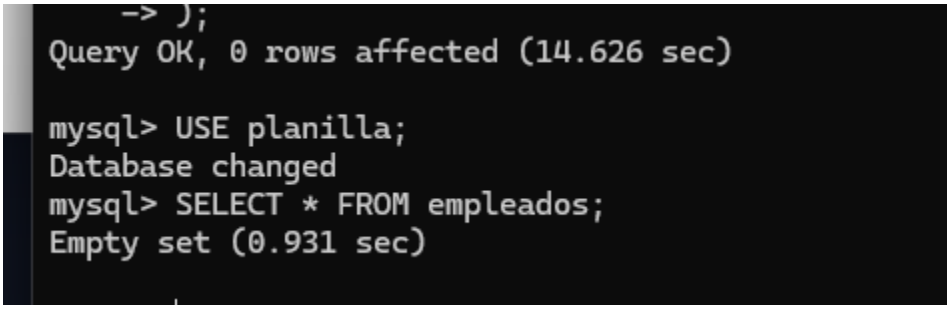
13. Configuración de la cadena de conexión

Utilizar la siguiente cadena de conexión:

string connectionString = "Server=localhost;Port=3307;Database=planilla;Uid=root;Pwd=erikaramirez123.;" ;

14. Prueba de Conexión

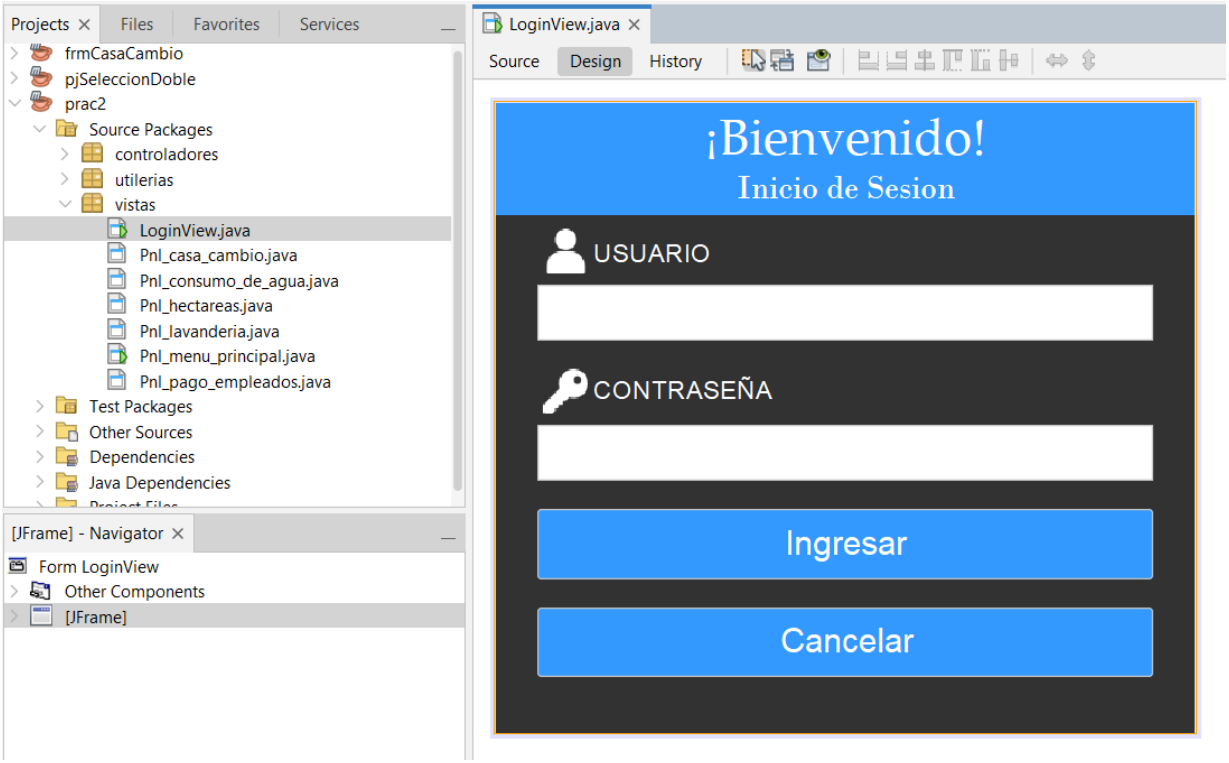
4. Ejecutar el programa C#
5. Verificar que no existan errores de conexión.
6. Validar acceso a la base de datos mediante consultas:



Descripción general

El sistema de login permite la autenticación de usuarios mediante la validación de credenciales almacenadas en una base de datos gestionada con MySQL.

La interfaz ha sido desarrollada en Java utilizando Apache NetBeans, integrando controles visuales y mecanismos de seguridad para mejorar la experiencia del usuario.



Interfaz del sistema

El sistema cuenta con los siguientes elementos:

- Campo de texto: **Usuario**
- Campo de texto: **Contraseña (oculta)**
- Botón: **Ingresar**
- Botón: **Cancelar**
- Checkbox: **Mostrar contraseña**
- Barra de progreso: **Indica acceso correcto**
- Etiqueta de error: **Muestra mensajes de validación**

1. Inicio del Sistema

2. Ejecutar la aplicación desde el entorno de desarrollo o archivo compilado.

3. Se desplegará la ventana principal de inicio de sesión.

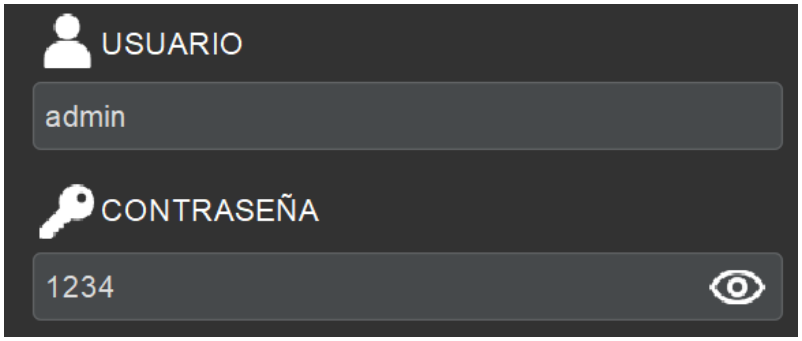
2. Ingreso de credenciales

1. Introducir el nombre de usuario en el campo correspondiente.
2. Introducir la contraseña en el campo de seguridad.

   La contraseña se encuentra oculta por defecto para mayor seguridad.

3. Mostrar u ocultar contraseña

1. Activar la casilla **“Mostrar contraseña”** para visualizar el contenido.
2. Desactivarla para volver a ocultarla.



4. Proceso de autenticación

1. Presionar el botón **Ingresar**.
2. El sistema realiza las siguientes acciones:
   - Valida que los campos no estén vacíos.
   - Se conecta a la base de datos.
   - Ejecuta una consulta segura mediante PreparedStatement.
   - Compara las credenciales con los registros existentes.
3. Si las credenciales son correctas:
   - Se activa una **barra de progreso** indicando acceso exitoso.

5. Manejo de errores

El sistema contempla distintos escenarios:

🔷 Campos vacíos

Credenciales incorrectas

- **Se muestra:**

Cancelación del sistema

1. Presionar el botón **Cancelar**.
2. El sistema solicitará confirmación:
    - "¿Seguro que deseas salir?"
3. Si se selecciona "Sí", la aplicación se cierra.

Funcionamiento técnico (resumen)

El sistema utiliza:

- Conexión JDBC mediante DriverManager
- Consultas seguras con PreparedStatement

- Validación de datos en tiempo real
- Manejo de excepciones SQL
- Interfaz gráfica con Swing

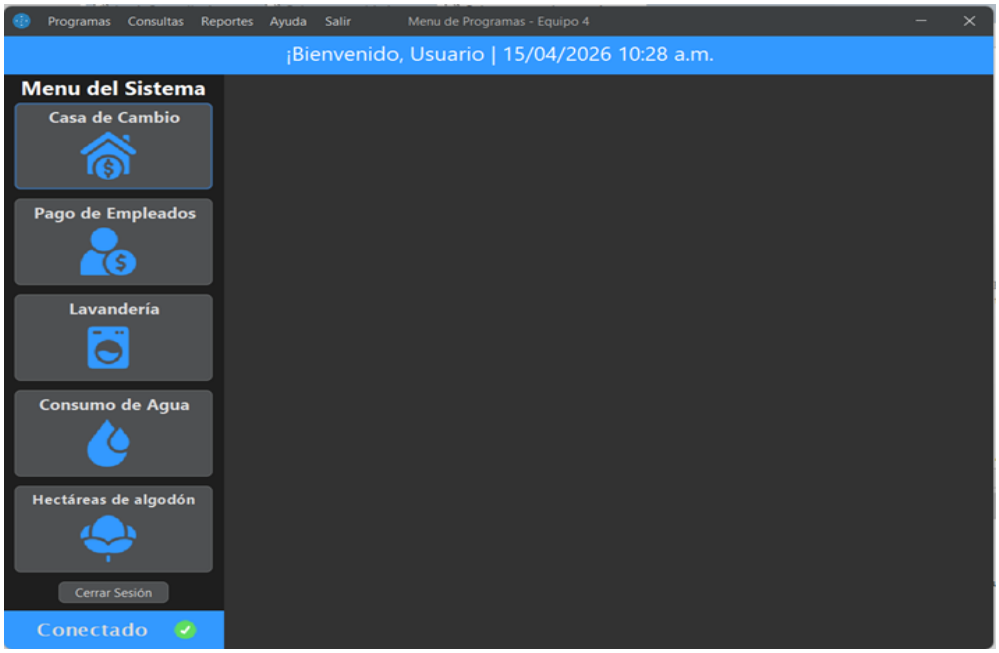
Indicador de acceso (Progress Bar)

- Se activa únicamente cuando el login es correcto.
- Muestra el progreso en porcentaje (0% a 100%).
- Representa visualmente el acceso al sistema.
- Cuando llega a 100% la progress bar, este accedera al menú con los 5 programas correspondientes

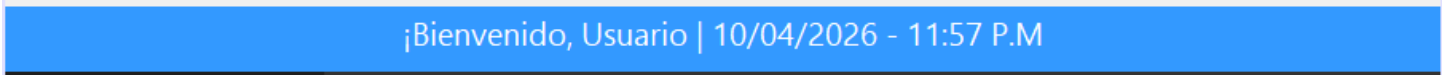
GUIA DE USO DEL SISTEMA DEL MENU

Descripción general

El menú mostrado corresponde a la interfaz principal de un sistema desarrollado en Java, el cual funciona como punto central de navegación para las diferentes funcionalidades del sistema. Su diseño está dividido en tres secciones principales:



Barra superior



En la parte superior se encuentra una barra de encabezado que incluye:

- Un mensaje de bienvenida al usuario.
- La fecha y hora actual del sistema.

Esta sección permite identificar rápidamente quién está utilizando el sistema y en qué momento.

Menú lateral (Panel de opciones)

Ubicado en el lado izquierdo, contiene las opciones principales del sistema organizadas de forma vertical. Cada opción está representada con un botón que incluye texto e icono para facilitar su identificación:

- **Casa de Cambio** → Acceso al módulo de operaciones de cambio de divisas.
- **Pago de Empleados** → Gestión de pagos al personal.
- **Lavandería** → Control de servicios relacionados con lavandería.
- **Consumo de Agua** → Registro o consulta de consumo.
- **Hectáreas de algodón** → Control o registro de producción agrícola.

También incluye:

- Botón de **Cerrar sesión**, para salir del sistema.
- Indicador de estado: **"Sin conexión"**, que muestra si el sistema tiene conexión activa con la base de datos.

Área de trabajo (Panel principal)

Es el espacio grande del lado derecho (zona oscura), donde se cargan las diferentes vistas o formularios dependiendo de la opción seleccionada en el menú lateral.
Esta área permite mantener una sola ventana mientras se navega entre módulos.

¿Cómo se hizo?

Este menú fue desarrollado utilizando:

Java Swing

Se utilizaron componentes gráficos de Java Swing, como:

- JFrame: Ventana principal del sistema.
- JPanel: Para dividir y organizar las secciones (menú lateral, encabezado y contenido).
- JButton: Para las opciones del menú.
- JLabel: Para textos como bienvenida y estado de conexión.
- JMenuBar: Barra de menú superior (Programas, Consultas, Reportes, etc.).

Resumen para manual

Este menú principal funciona como la interfaz central del sistema, permitiendo al usuario acceder de manera rápida y organizada a los diferentes módulos mediante un panel lateral. Fue desarrollado en Java utilizando Swing dentro del entorno NetBeans, empleando paneles dinámicos para cargar las distintas vistas sin necesidad de abrir nuevas ventanas.

GUIA DEL USO DE LOS 5 PROGRAMAS

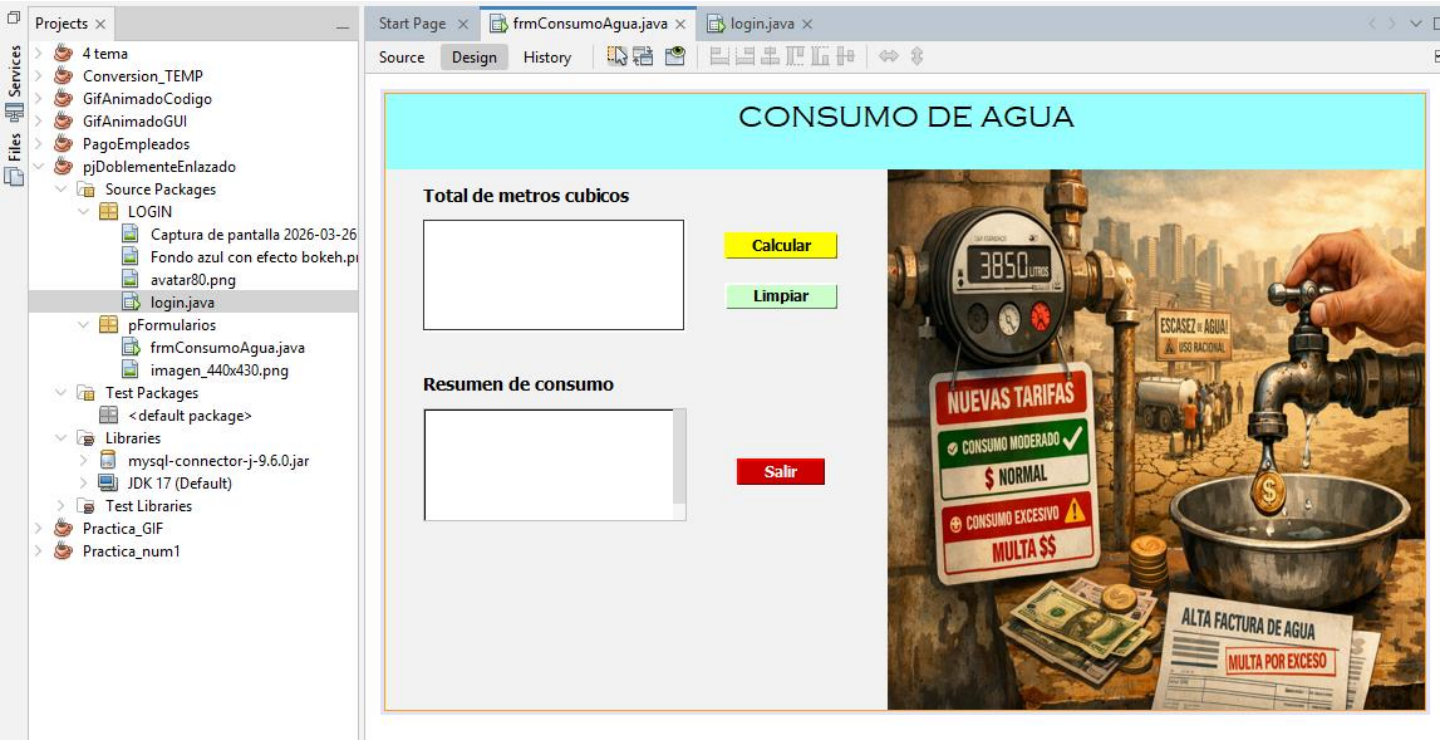
PROGRAMA 1: CONSUMO DEL AGUA (Realizado por: Guillermo Cambrón Escobar. Realizado con JDK 17 en Neatbeans)

Descripción general

El presente programa corresponde a uno de los cinco módulos funcionales integrados dentro de un sistema general que inicia con un proceso de autenticación (login).

Una vez validado el usuario en el sistema, se despliega un menú principal que permite acceder a distintos programas conectados a una base de datos central gestionada mediante MySQL.

El módulo de **Consumo de Agua** permite calcular el monto a pagar en función del consumo registrado, aplicar impuestos cuando corresponda y almacenar los resultados en la base de datos para su posterior consulta.



1. **Interfaz del sistema**

El módulo cuenta con los siguientes componentes:

- Campo de entrada: **Consumo en metros cúbicos (m^3)**
- Botón: **Calcular**

- Botón: **Limpiar**
- Botón: **Salir**
- Área de texto: **Resumen de consumo y registros almacenados**

2. Ingreso de datos

- Introducir el valor del consumo de agua en metros cúbicos en el campo correspondiente.

El sistema valida que el campo no esté vacío ni contenga valores negativos.

3. Proceso de cálculo

1. Presionar el botón **Calcular**.
2. El sistema realiza automáticamente:

🔷 **Validaciones:**

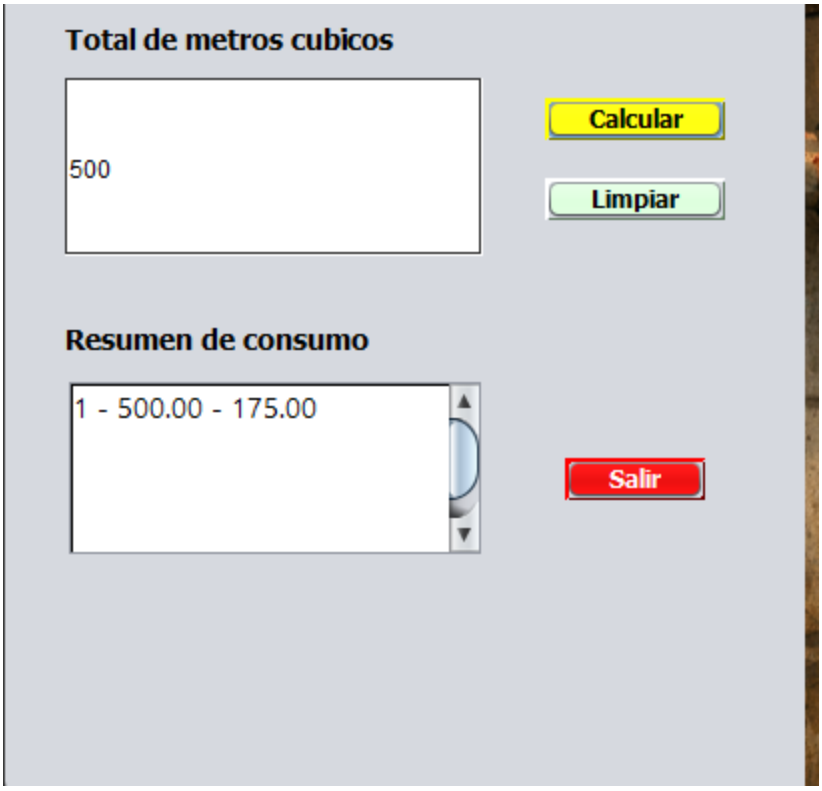
- Verifica que el campo no esté vacío.
- Verifica que el valor sea numérico y positivo.

🔷 **Cálculo del monto:**

- Menor a 100 m³ → tarifa de 0.15
- Entre 100 y 499 m³ → tarifa de 0.20
- Entre 500 y 999 m³ → tarifa de 0.35
- Mayor o igual a 1000 m³ → tarifa de 0.80

🔷 **Aplicación de impuesto:**

- Si el monto es mayor a $600 → se aplica un 2.5% adicional



4. Registro en base de datos

El sistema realiza automáticamente:

- Conexión a la base de datos
- Inserción de los datos en la tabla consumo_agua
- Consulta de todos los registros existentes

Esto permite llevar un control histórico de consumos y pagos.

5. Manejo de errores

El sistema contempla los siguientes casos:

🔹 **Campo vacío**

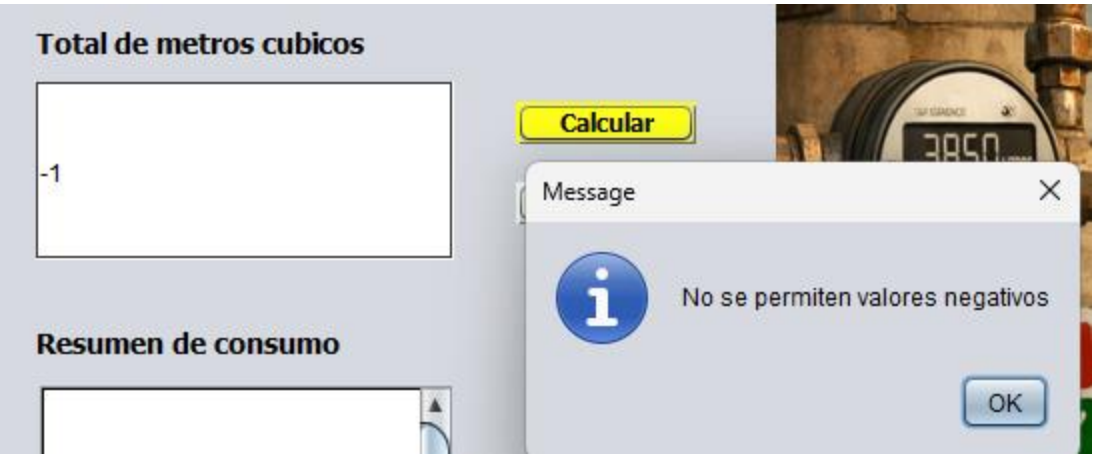
“Ingresa un valor de consumo”

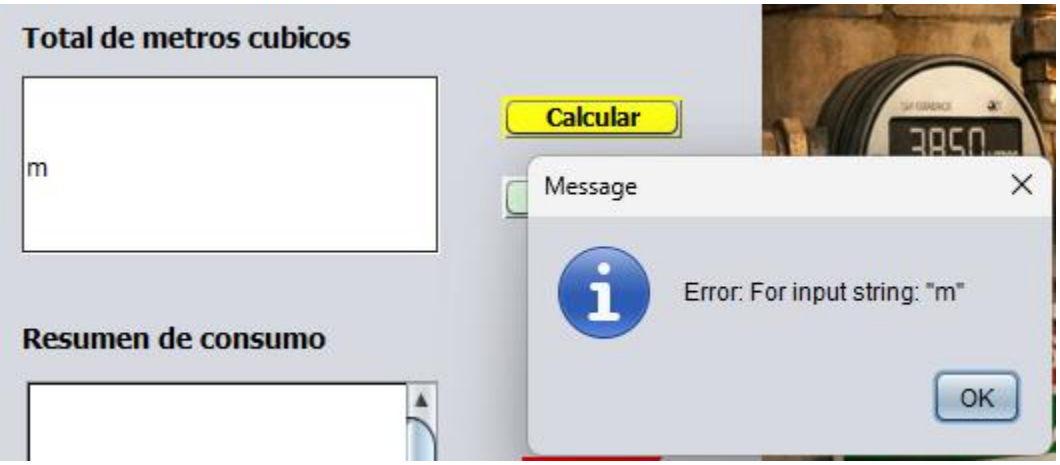
🔹 **Valor negativo**

“No se permiten valores negativos”

🔹 **Error en ejecución**

- Se muestra un mensaje emergente con el detalle del error





6. Limpieza de datos

1. Presionar el botón **Limpiar**.
2. El sistema:

- Borra el campo de consumo
- Limpia el área de resultados
- Regresa el cursor al campo principal

7. Salida del Sistema

1. Presionar el botón **Salir**.
2. El sistema solicitará confirmación.
3. Si se acepta, se cerrará la aplicación.

```
run:
BUILD SUCCESSFUL (total time: 4 seconds)
```

8. Funcionamiento técnico (resumen)

El módulo implementa:

- Conexión JDBC mediante DriverManager
- Inserciones con PreparedStatement
- Consultas con Statement
- Manejo de excepciones
- Interfaz gráfica con Swing

PROGRAMA 2: NIVEL DE AGUA EN LAVANDERÍA

GENESIS NERINA GARCIA CONTRERAS

Realizado por Genesis Nerina García Contreras con Visual Studio Community en lenguaje C# (.NET Windows Forms)

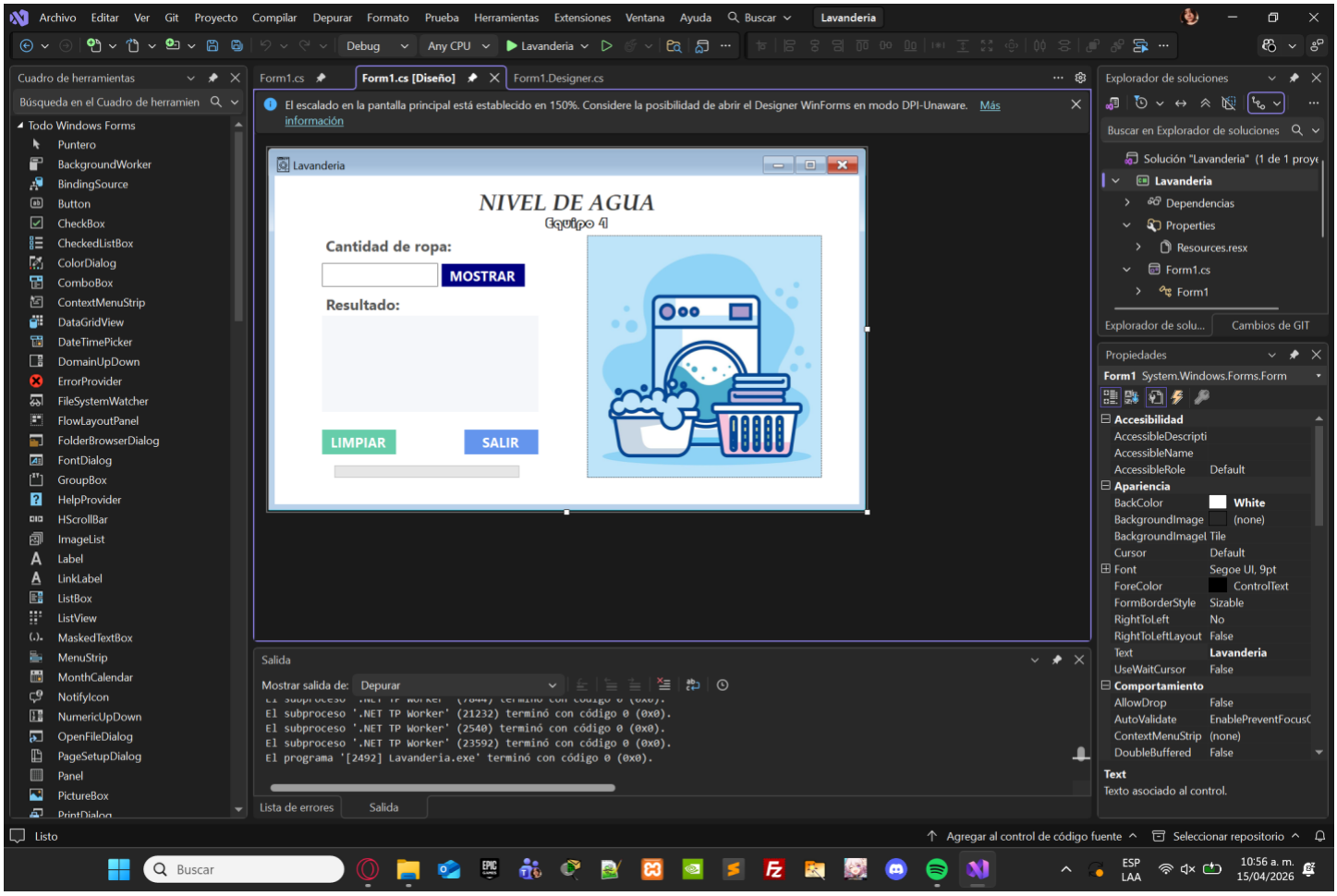
Descripción

El presente programa forma parte de un conjunto de aplicaciones diseñadas para automatizar procesos mediante el uso de interfaces gráficas.

Este módulo está enfocado en el cálculo del nivel de agua requerido en una lavadora, tomando como base la cantidad de ropa ingresada en kilogramos. A partir de este valor, el sistema determina el nivel adecuado de llenado y calcula una estimación de agua necesaria para el proceso de lavado.

Además, el sistema incorpora mecanismos de validación para garantizar que los datos ingresados sean correctos, evitando errores durante la ejecución. Los resultados obtenidos pueden almacenarse en una base de datos, lo que permite llevar un registro histórico de las operaciones realizadas.

De esta manera, el programa no solo cumple con una función de cálculo, sino que también contribuye a la organización y control de la información generada durante su uso.



Interfaz del sistema

El módulo cuenta con los siguientes componentes:

- Campo de entrada: Kilos de ropa (kg)
- Botón: Mostrar
- Botón: Limpiar
- Botón: Salir
- Área de texto: Resultado del cálculo
- Barra de progreso: Mide el nivel de agua usada

Ingreso de datos

El usuario debe:

- Introducir la cantidad de ropa en kilogramos
- El sistema valida que:
- El campo no esté vacío
- El valor sea numérico
- El valor sea positivo

Proceso de cálculo

Al presionar el botón Mostrar, el sistema realiza automáticamente:

Validaciones

- Verifica que el valor sea numérico
- Verifica que no exceda el máximo permitido (30 kg)

Determinación del nivel de agua

Según los kilos ingresados:

- Menor a 8 kg → Nivel MÍNIMO
- Entre 8 y 14 kg → Nivel MEDIO
- Entre 15 y 21 kg → Nivel ALTO
- Entre 22 y 30 kg → Nivel MÁXIMO
- Mayor a 30 kg → Error (exceso de peso)

Lavanderia

NIVEL DE AGUA

Equipo 4

Cantidad de ropa:

5 MOSTRAR

Resultado:

Kilos: 5
Nivel: MINIMO
Agua: 50 litros

LIMPIAR SALIR

Lavanderia

NIVEL DE AGUA

Equipo 4

Cantidad de ropa:

12 MOSTRAR

Resultado:

Kilos: 12
Nivel: MEDIO
Agua: 120 litros

LIMPIAR SALIR

Lavanderia

NIVEL DE AGUA

Equipo 4

Cantidad de ropa:

20 MOSTRAR

Resultado:

Kilos: 20
Nivel: ALTO
Agua: 200 litros

LIMPIAR SALIR

Lavanderia

NIVEL DE AGUA

Equipo 4

Cantidad de ropa:

27 MOSTRAR

Resultado:

Kilos: 27
Nivel: MAXIMO
Agua: 270 litros

LIMPIAR SALIR

Cálculo de agua

El sistema calcula la cantidad de agua requerida:

Litros = Kilos × 10

Registro en base de datos

El sistema realiza automáticamente:

Conexión a la base de datos mediante XAMPP

Inserción de los datos en la tabla historial

Almacenamiento de:

- Kilos
- Nivel
- Litros
- Fecha

Esto permite llevar un control histórico de los lavados.

El sistema contempla los siguientes casos:

- Valor inválido

“Ingresa un número válido”

- Exceso de peso

“Capacidad máxima excedida (30 kg)”

Limpieza de datos

Al presionar el botón Limpiar:

El sistema:

- Borra el campo de entrada
- Limpia el área de resultados
- Regresa el cursor al campo principal

Salida del sistema

Al presionar el botón Salir:

- El sistema solicita confirmación
- Si el usuario acepta, se cierra la aplicación

Funcionamiento técnico

El módulo implementa:

- Interfaz gráfica con Windows Forms
- Programación en C#
- Uso de estructuras condicionales (if-else)
- Validación de datos
- Conexión a base de datos mediante XAMPP
- Inserciones con comandos SQL
- Manejo de excepciones (try-catch)

PROGRAMA DE HECTÁREAS DE ALGODÓN Y MAÍZ

LISSET ISIDRO SANCHEZ

Desarrollado en Java con Swing y MySQL

Descripción general

El presente programa forma parte de un sistema integral compuesto por distintos módulos conectados a una base de datos central mediante MySQL.

Este módulo permite al usuario calcular la distribución de hectáreas destinadas al cultivo de algodón y maíz, determinar el costo de producción de cada uno, identificar cuál cultivo genera mayor costo y almacenar los resultados en la base de datos para su control y análisis.

Interfaz del sistema

El sistema cuenta con los siguientes componentes:

• Campo de entrada: Hectáreas de algodón
• Campo de entrada: Hectáreas de maíz
• Botón: Calcular
• Botón: Limpiar
• Botón: Salir
• Área de textos: Resultados del cálculo

Ingreso de datos

• El usuario debe ingresar:

- Cantidad de hectáreas de algodón
- Cantidad de hectáreas de maíz

🔷 Validaciones realizadas por el sistema:

- No permitir campos vacíos
- No permitir valores negativos
- Solo aceptar valores numéricos

Proceso de cálculo

1. Presionar el botón **Calcular**
2. El sistema realiza automáticamente:

Cálculos principales

• Total de hectáreas:

```
Total = Algodón + Maíz
```

• Costo de producción:

- Algodón → $3.00 por hectárea
- Maíz → $5.00 por hectárea

```
Costo Algodón = hectáreas * 3.00
Costo Maíz = hectáreas * 5.00
```

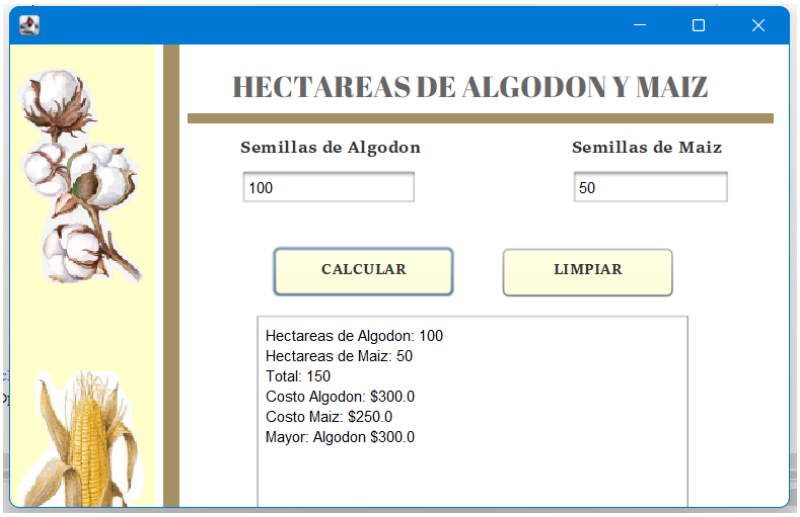
Determinación del cultivo mayor

El sistema compara los costos:

- Si el costo del algodón es mayor → se selecciona algodón
- Si el costo del maíz es mayor → se selecciona maíz

Se muestra:

- Nombre del cultivo con mayor costo
- Valor del costo mayor



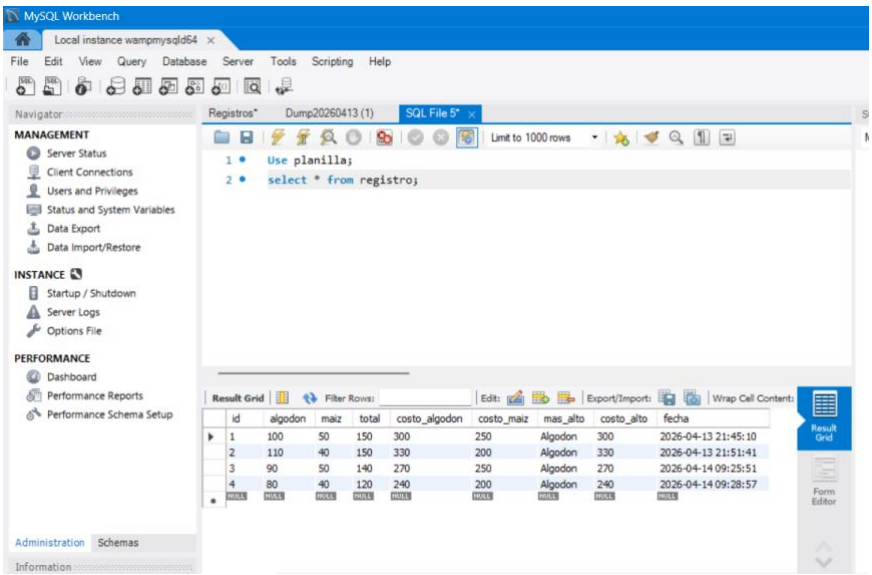
Registro en base de datos

El sistema realiza automáticamente:

- Conexión a la base de datos mediante JDBC
  Inserción de datos en la tabla registro
  Almacenamiento de:

- Hectáreas de algodón
- Hectáreas de maíz
- Total
- Costos
- Cultivo mayor
- Costo mayor

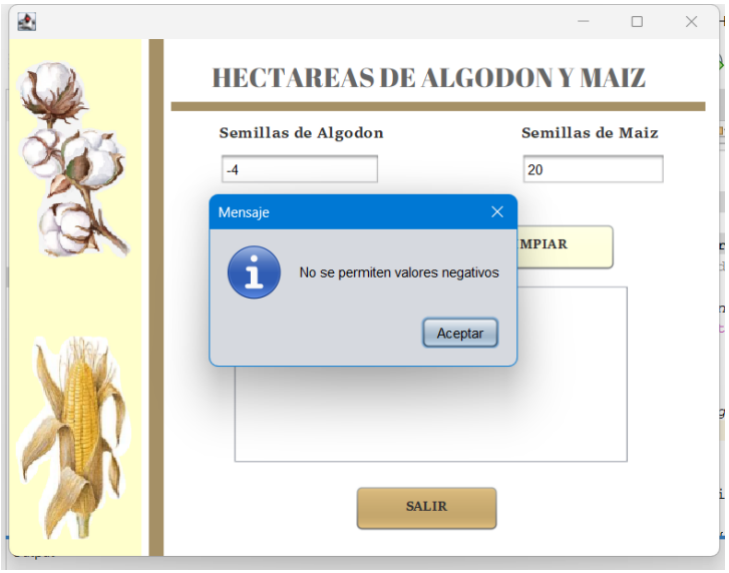
Esto permite llevar un historial de cálculos realizados.



Manejo de errores

El sistema contempla los siguientes casos:

- Campo vacío
  “Ingresa todos los valores”
- Valor negativo
  “No se permiten valores negativos”
- Error en base de datos
  Se muestra un mensaje emergente con el detalle del error
- Error de ejecución
  Se notifica mediante ventana emergente (JOptionPane)



Limpieza de datos

1. Presionar el botón **Limpiar**
   El sistema realiza:

- Limpieza de los campos de entrada
- Limpieza del área de resultados
- Posiciona el cursor en el primer campo

Salida del sistema

1. Presionar el botón **Salir**
2. El sistema solicita confirmación
3. Si el usuario acepta, la aplicación se cierra

FUNCIONAMIENTO TECNICO

El módulo implementa:

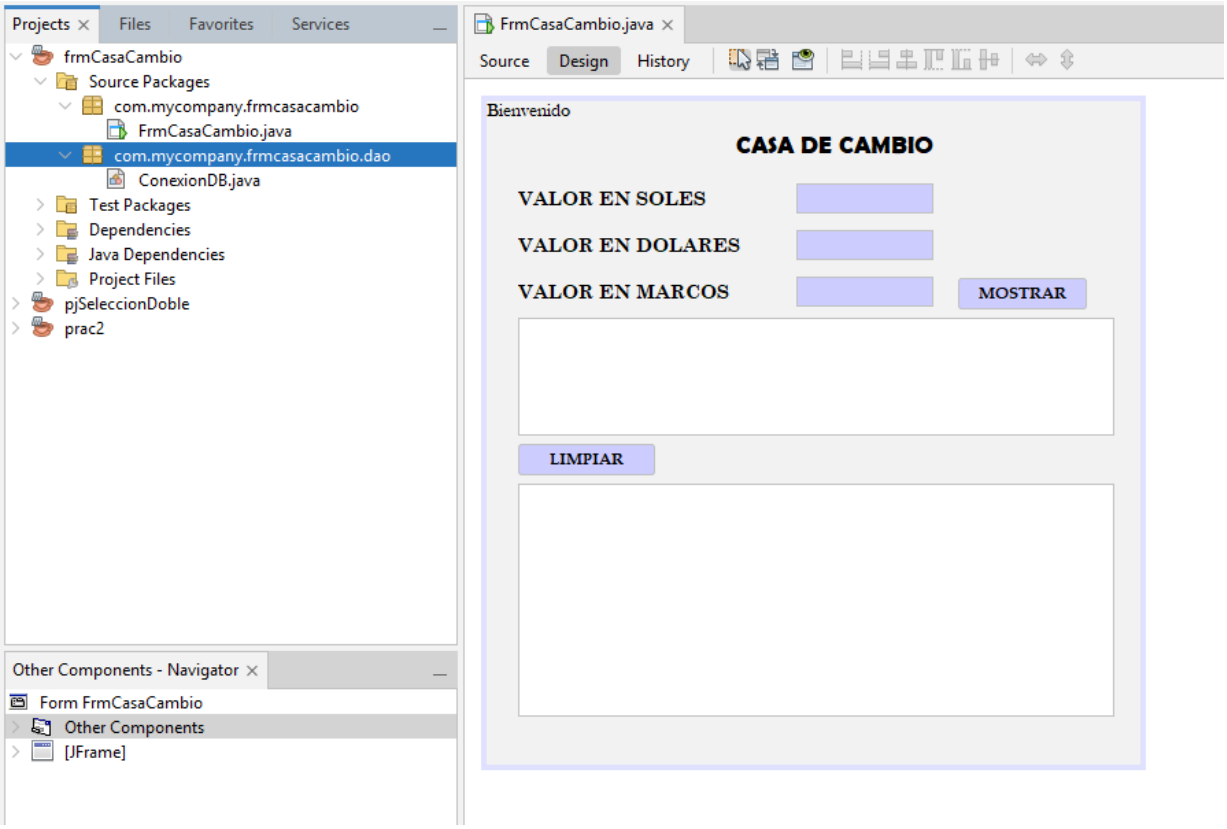
- Interfaz gráfica con Java Swing
- Conexión a base de datos mediante JDBC
- Uso de PreparedStatement para inserciones seguras
- Manejo de excepciones (try-catch)
- Validaciones de entrada de datos
- Lógica de negocio para cálculos agrícolas

Programa de "Casa de Cambio"

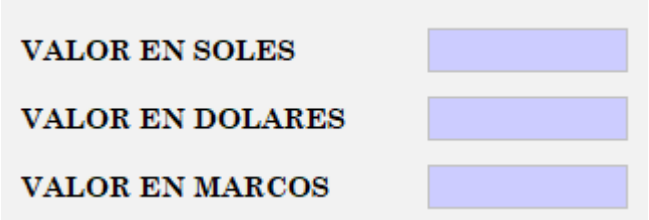
EZRY ALEXANDRA CONRERAS

La interfaz mostrada corresponde al módulo **Casa de Cambio**, el cual permite capturar y visualizar valores de distintas monedas dentro del sistema. Este formulario facilita el ingreso de datos y su posterior procesamiento o visualización.

El diseño está estructurado de manera simple y funcional, dividido en las siguientes secciones:



Área de captura de datos



Se presentan tres campos de entrada donde el usuario puede ingresar valores:

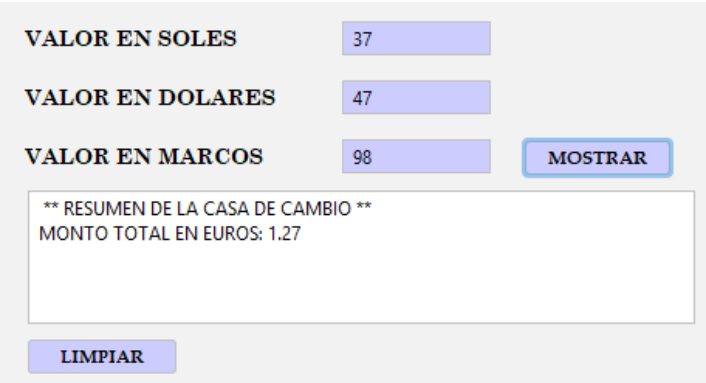
- **Valor en soles** → Campo para moneda peruana.
- **Valor en dólares** → Campo para moneda estadounidense.
- **Valor en marcos** → Campo para otra divisa (posiblemente marco alemán o de referencia).

Estos campos están implementados mediante componentes de texto que permiten al usuario ingresar datos numéricos.

Botones de acción

- **MOSTRAR**
  Permite procesar o mostrar los valores ingresados. Generalmente, este botón ejecuta algún evento que toma los datos de los campos y los presenta en el área de resultados.
- **LIMPIAR**
  Se utiliza para borrar los datos ingresados en los campos y limpiar las áreas de salida.

Ingreso de datos

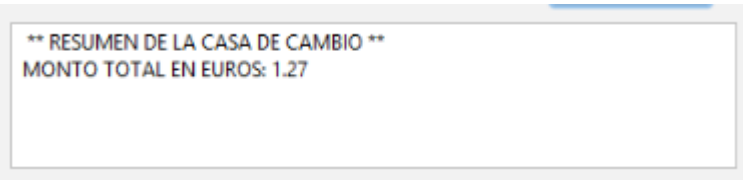


El usuario capturó los siguientes valores en los campos correspondientes:

- **Valor en soles:** 37
- **Valor en dólares:** 47
- **Valor en marcos:** 98

Estos datos representan cantidades en diferentes monedas que serán utilizadas para realizar un cálculo o conversión.

Visualización de resultados



En el área de resultados se muestra el siguiente mensaje:

- **"RESUMEN DE LA CASA DE CAMBIO"**
- **"MONTO TOTAL EN EUROS: 1.27"**

Esto indica que el sistema realizó una conversión de las cantidades ingresadas a una moneda común (euros), mostrando el total calculado.

Tabla de datos

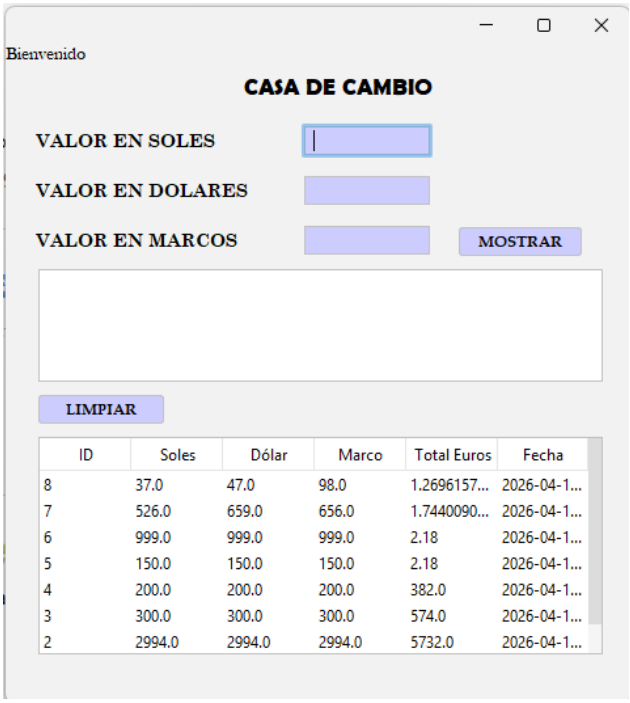
Se observa una tabla que organiza la información en forma de filas y columnas. Cada fila representa un registro distinto guardado en el sistema.

ID	Soles	Dólar	Marco	Total Euros	Fecha
8	37.0	47.0	98.0	1.2696157...	2026-04-1...
7	526.0	659.0	656.0	1.7440090...	2026-04-1...
6	999.0	999.0	999.0	2.18	2026-04-1...
5	150.0	150.0	150.0	2.18	2026-04-1...
4	200.0	200.0	200.0	382.0	2026-04-1...
3	300.0	300.0	300.0	574.0	2026-04-1...
2	2994.0	2994.0	2994.0	5732.0	2026-04-1...

Las columnas son las siguientes:

- **ID** → Identificador único de cada registro.
- **Soles** → Cantidad ingresada en moneda peruana.
- **Dólar** → Cantidad ingresada en dólares.
- **Marco** → Cantidad ingresada en marcos.
- **Total Euros** → Resultado de la conversión total a euros.
- **Fecha** → Fecha en que se realizó el registro.

Funcionalidad



Esta tabla permite:

- Consultar operaciones anteriores.
- Llevar un control histórico de las conversiones realizadas.
- Ver la evolución o variación de los cálculos en diferentes momentos.

Los datos que se muestran probablemente provienen de una **base de datos**, ya que incluyen múltiples registros con fechas distintas.

Relación con el sistema

Esta sección está conectada con el módulo de captura (Casa de Cambio), ya que:

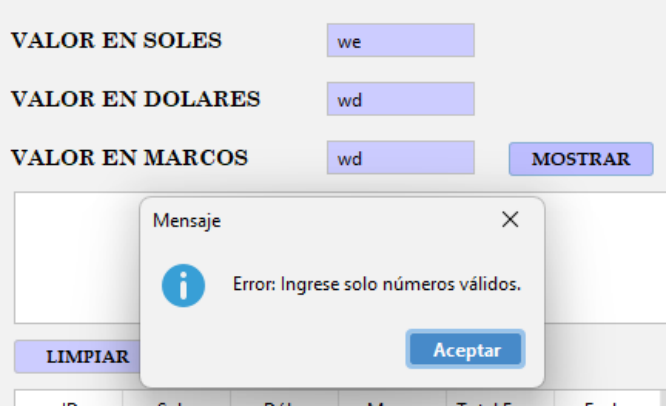
- Cada vez que el usuario realiza una operación y la guarda, se genera un nuevo registro en esta tabla.
- La información se recupera mediante consultas (posiblemente usando una clase como `ConexionDB`).

Componentes utilizados

Para esta interfaz se emplearon:

- `JTable` → Para mostrar los datos en forma tabular.
- `DefaultTableModel` → Para gestionar la información de la tabla.
- Conexión a base de datos → Para obtener los registros almacenados.

Ingreso incorrecto de datos

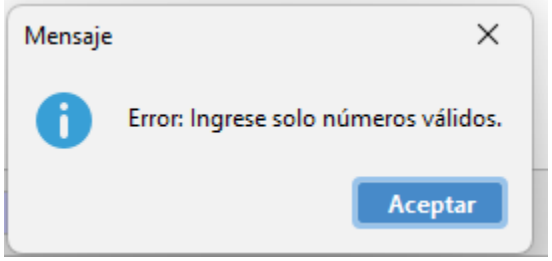


En los campos de captura se ingresaron valores no numéricos:

- **Valor en soles:** "we"
- **Valor en dólares:** "wd"
- **Valor en marcos:** "wd"

Estos valores no son válidos, ya que el sistema espera únicamente números para realizar los cálculos.

Mensaje de error



El sistema muestra una ventana emergente (diálogo) con el siguiente mensaje:

“Error: Ingrese solo números válidos.”

Este mensaje cumple con las siguientes funciones:

- Informar al usuario sobre el error cometido.
- Indicar cómo corregirlo.
- Prevenir fallos en la ejecución del programa.

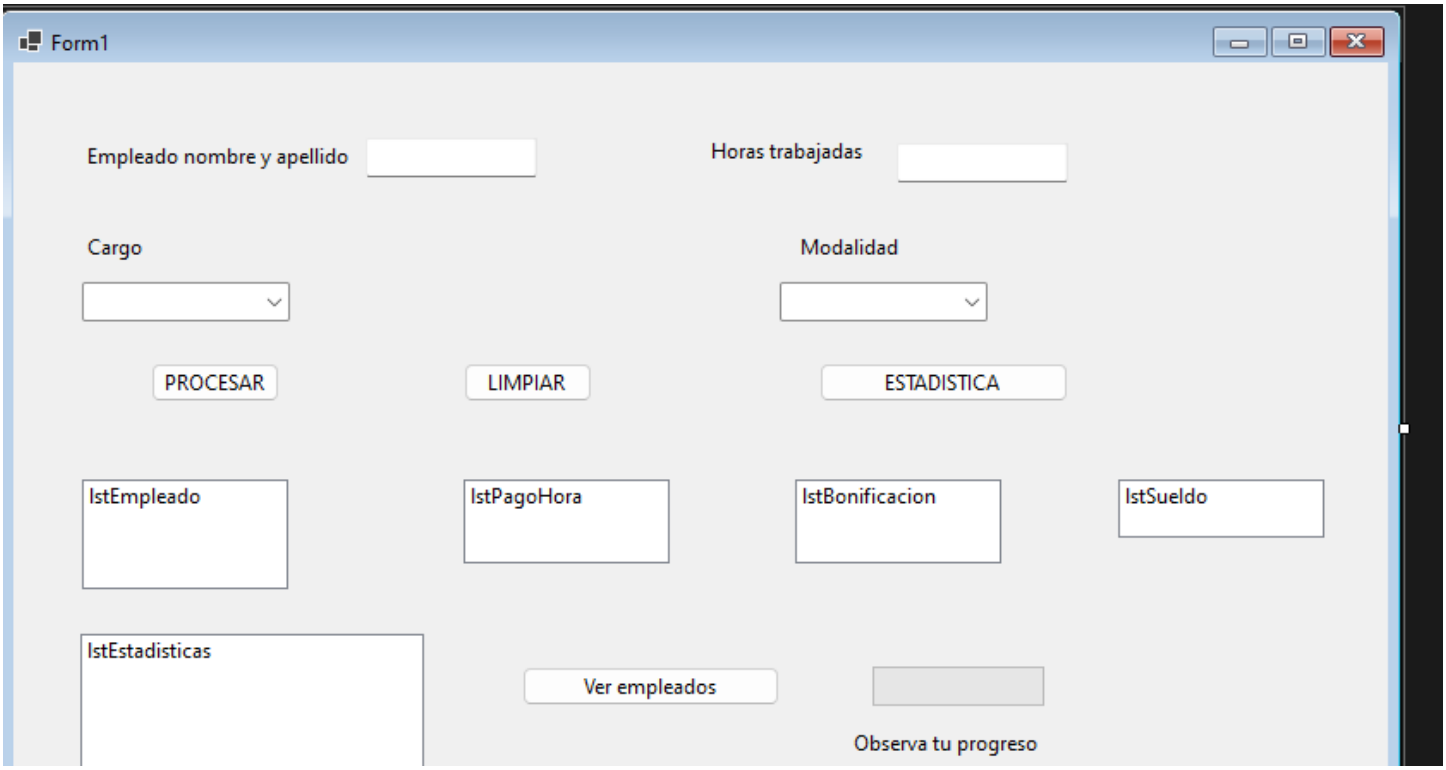
Esta sección muestra el mecanismo de validación del sistema, el cual impide que el usuario ingrese datos no numéricos. En caso de error, se despliega un mensaje informativo que guía al usuario a introducir valores correctos, garantizando así la integridad de los cálculos.

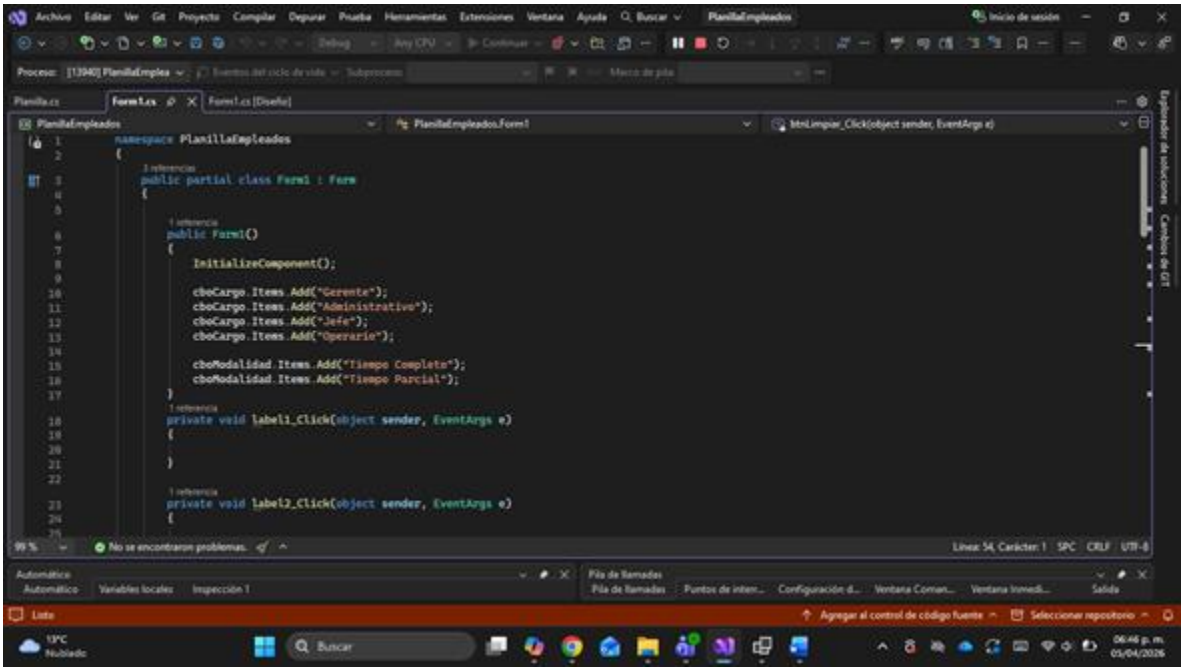
PROGRAMA DE PAGO DE EMPLEADOS

ERIKA RAMIREZ CHAVARRIA

Desarrollado en Visual Studio en C# y MySQL command Line client

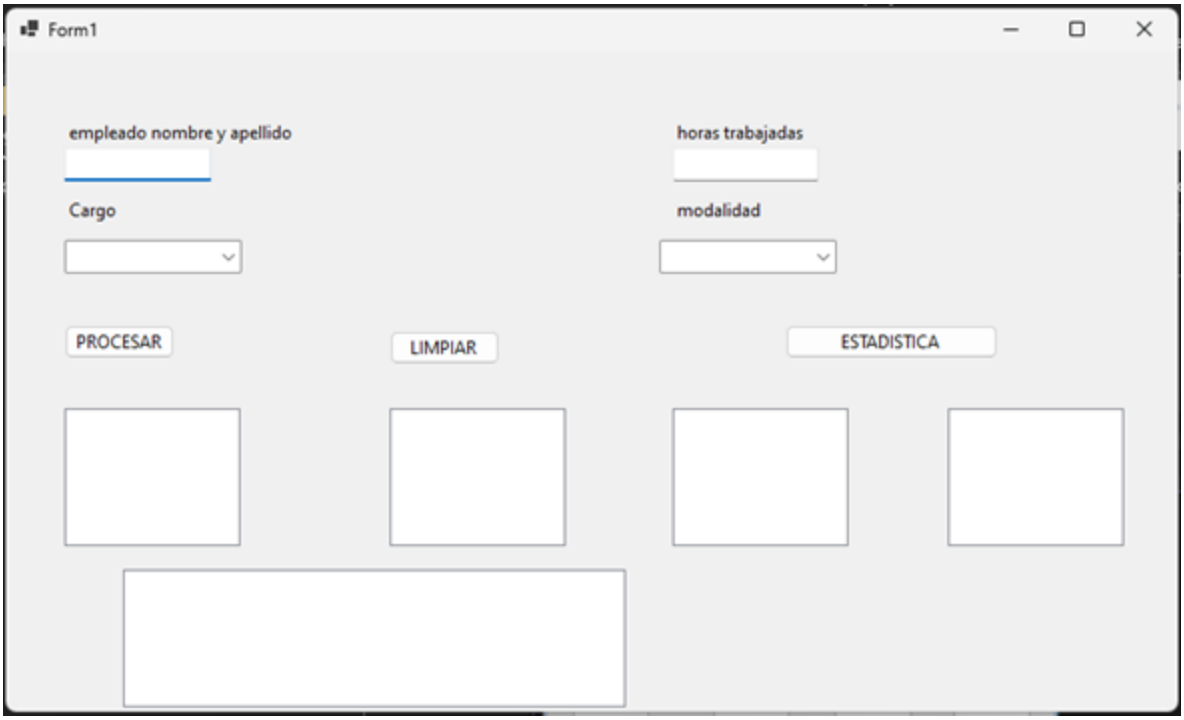
La interfaz muestra los labels para guiar todo, “Empleado nombre y apellido”, otro para las “Horas trabajadas”, otro para “Cargo” y el último para “Modalidad”. Debajo tengo mis textboxes y combos donde meto los datos, y luego la fila de botones: el de PROCESAR que hace los cálculos, el de LIMPIAR que borra todo, el de ESTADISTICA que me muestra datos generales y el de Ver empleados que lista lo que ya guardé. Más abajo se ven las listas (lstEmpleado, lstPagoHora, lstBonificacion, lstSueldo, lstEstadisticas) donde se van mostrando los resultados, y una barra de progreso con su label “Observa tu progreso” para que se vea más interactivo.


Form1
Empleado nombre y apellido
Horas trabajadas
Cargo
Modalidad
PROCESAR
LIMPIAR
ESTADISTICA
lstEmpleado
lstPagoHora
lstBonificacion
lstSueldo
lstEstadisticas
Ver empleados
Observa tu progreso


namespace PlanillaEmpleados
{
public partial class Form1 : Form
{
public Form1()
{
InitializeComponent();
cboCargo.Items.Add("Gerente");
cboCargo.Items.Add("Administrativo");
cboCargo.Items.Add("Jefe");
cboCargo.Items.Add("Operario");
cboModalidad.Items.Add("Tiempo Completo");
cboModalidad.Items.Add("Tiempo Parcial");
}
private void label1_Click(object sender, EventArgs e)
{
}
private void label2_Click(object sender, EventArgs e)
{

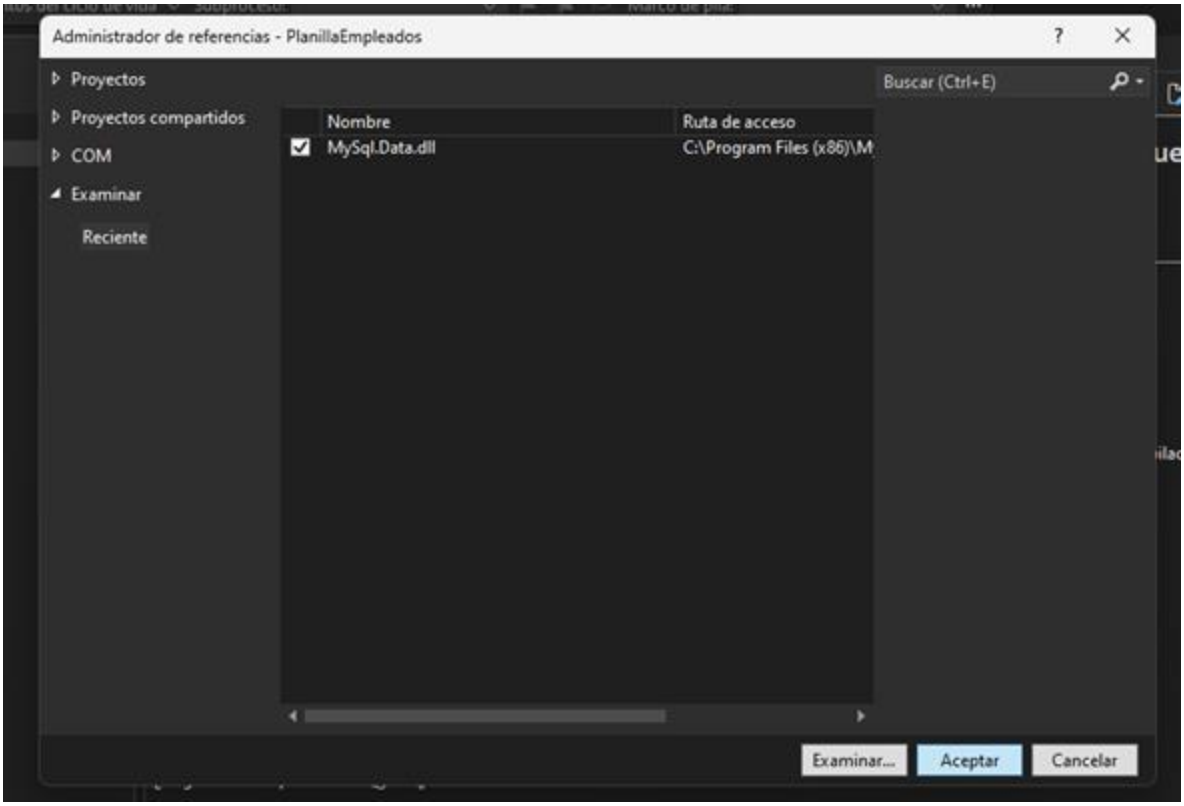
```
        }

        1 referencia
        private void btnProcesar_Click(object sender, EventArgs e)
        {
            string nombre = txtEmpleado.Text;

            int horas = int.Parse(txtHoras.Text);

            int cargo = cboCargo.SelectedIndex;
            int modalidad = cboModalidad.SelectedIndex;

            Planilla p = new Planilla(horas, cargo, modalidad);

            lstEmpleado.Items.Add(nombre);
            lstPagoHora.Items.Add(p.PagoHora());
            lstBonificacion.Items.Add(p.BoniFicacion());
            lstSueldo.Items.Add(p.SueldoFinal());
        }

        1 referencia
        private void btnLimpiar_Click(object sender, EventArgs e)
        {

            txtEmpleado.Clear();
            txtHoras.Clear();

            cboCargo.SelectedIndex = -1;
```

```
        }

        1 referencia
        private void btnLimpiar_Click(object sender, EventArgs e)
        {

            txtEmpleado.Clear();
            txtHoras.Clear();

            cboCargo.SelectedIndex = -1;
            cboModalidad.SelectedIndex = -1;

            txtEmpleado.Focus();
        }

        1 referencia
        private void btnEstadisticas_Click(object sender, EventArgs e)
        {
            lstEstadisticas.Items.Add("Total empleados: " + lstEmpleado.Items.Count);
        }
    }
}
```

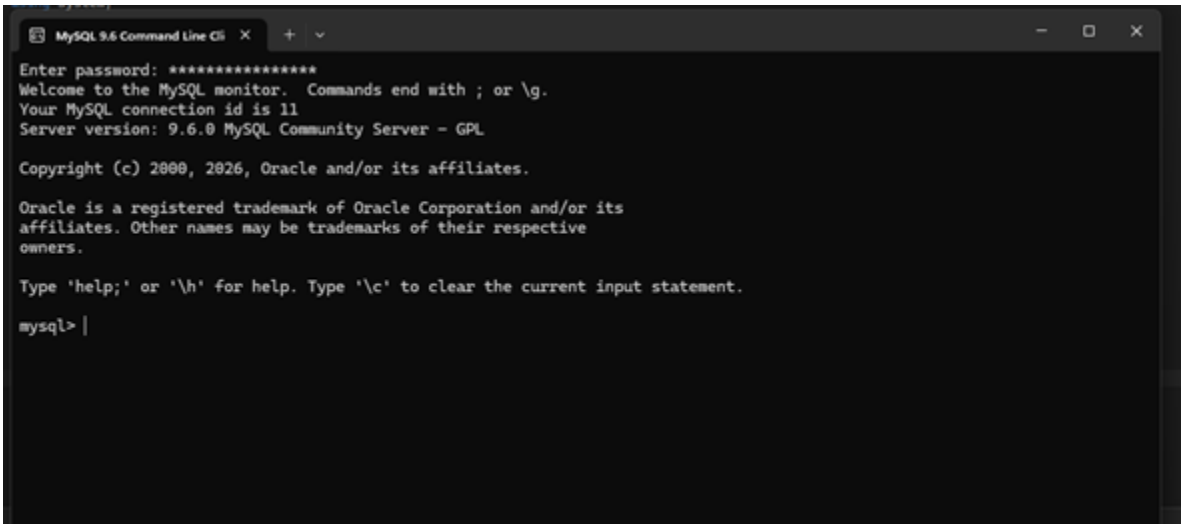


Continuamos con la conexión a la base de datos, descargando el driver que previamente se encontraba en teams

Abrimos el administrador de referencias y añadimos el driver para MySQL que descargamos e instalamos



Buscamos MySQL Command line client en el buscador de aplicaciones de nuestro equipo e ingresamos la contraseña que nos pidió al momento de instalar MySQL



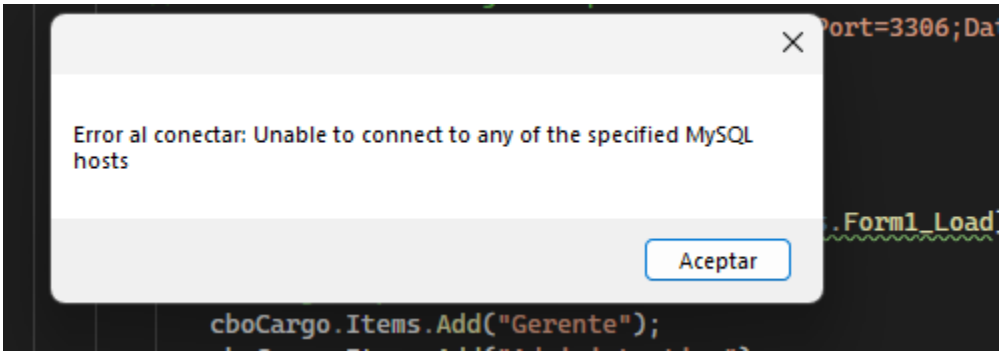
Con los comando creamos una base de datos llamada Planilla y posteriormente las columnas de la tabla, que sean Id, nombre, horas, cargo, modalidad, pagoHora, bonificación y sueldoFinal

```
MySQL 9.6 Command Line Cli

mysql> -- Crear la base de datos
Query OK, 0 rows affected (0.007 sec)

mysql> CREATE DATABASE planilla;
Query OK, 1 row affected (8.822 sec)

mysql>
mysql> -- Usar la base de datos
Query OK, 0 rows affected (0.002 sec)

mysql> USE planilla;
Database changed
mysql>
mysql> -- Crear la tabla empleados
Query OK, 0 rows affected (0.004 sec)

mysql> CREATE TABLE empleados (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     nombre VARCHAR(100) NOT NULL,
    ->     horas INT NOT NULL,
    ->     cargo VARCHAR(50) NOT NULL,
    ->     modalidad VARCHAR(50) NOT NULL,
    ->     pagoHora DECIMAL(10,2),
    ->     bonificacion DECIMAL(10,2),
    ->     sueldoFinal DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (14.626 sec)

mysql>
```

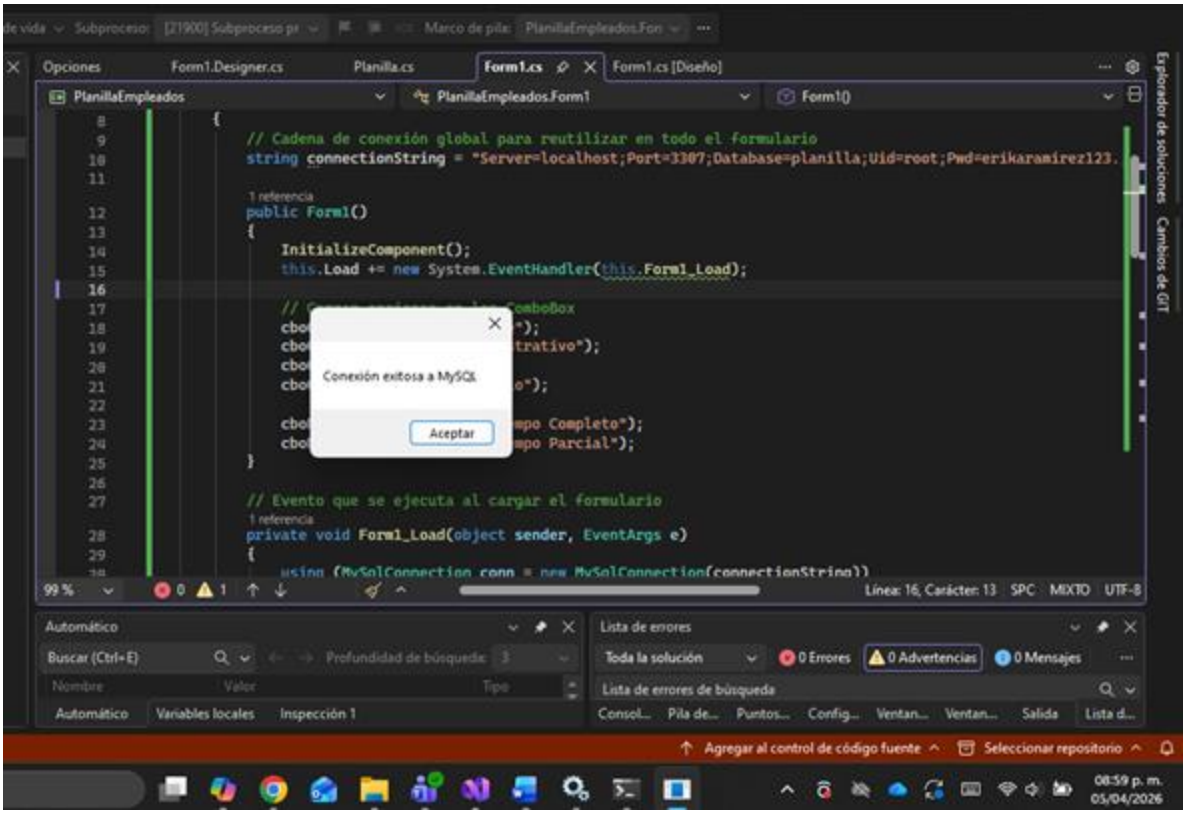
Hacemos la conexión a la base de datos con un String

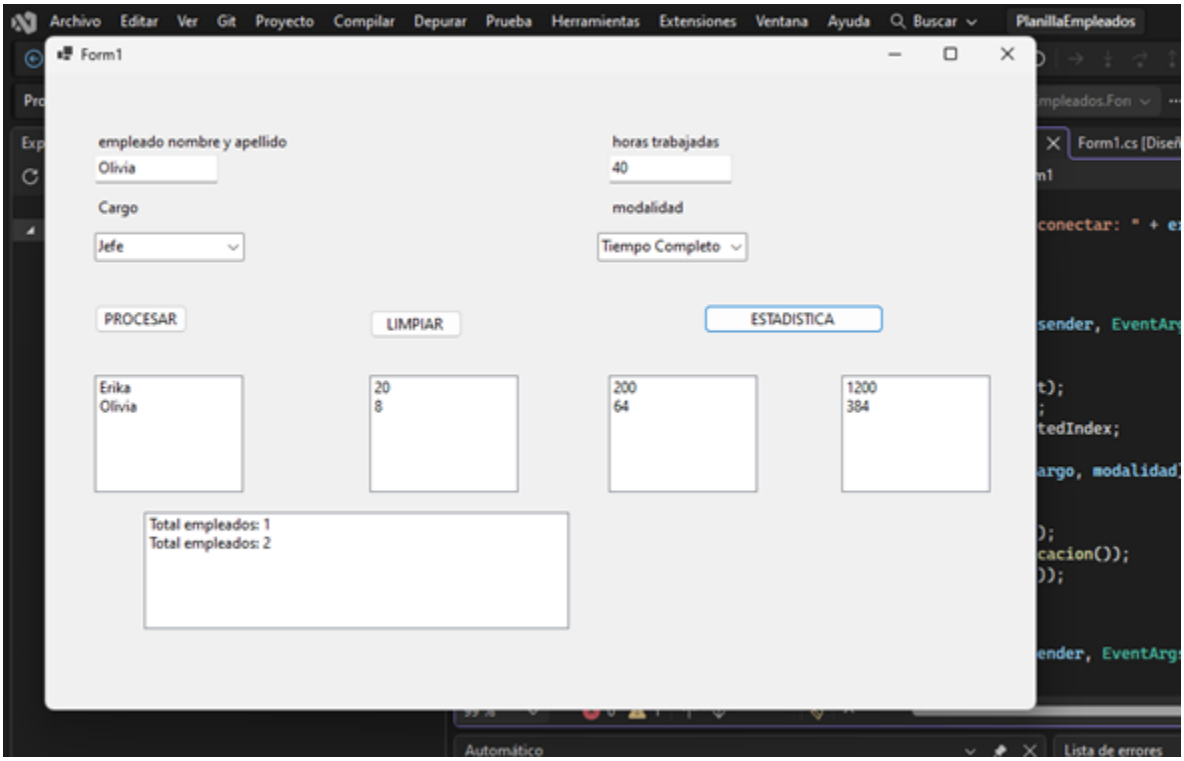
```
3 referencias
public partial class Form1 : Form
{
    // Cadena de conexión global para reutilizar en todo el formulario
    string connectionString = "Server=localhost;Port=3306;Database=planilla;Uid=root;Pwd=erikaramirez123.$;";

    1 referencia
    public Form1()
    {
        InitializeComponent();
```

Mi erro fue que al momento de teclear, puse un símbolo “$” al final de la linea



Corregimos ese error, en el puerto normalmente es el 3306, pero durante la instalación de MySQL yo designe el puerto 3307 asi que eso depende de la configuración al momento de instalar

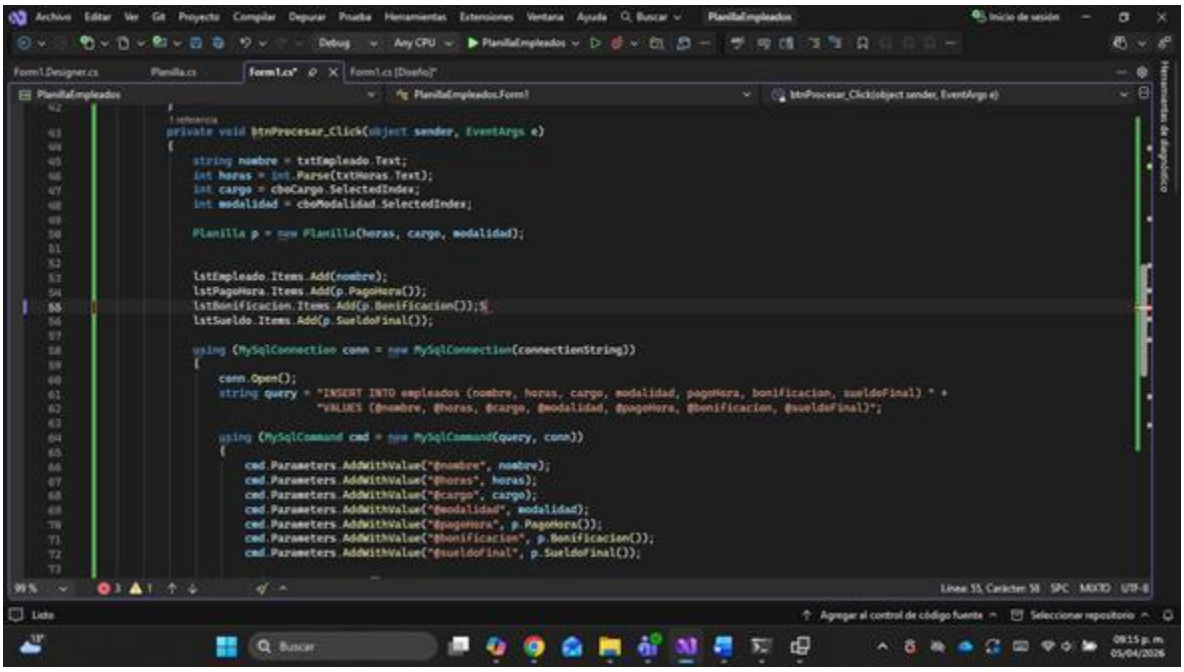




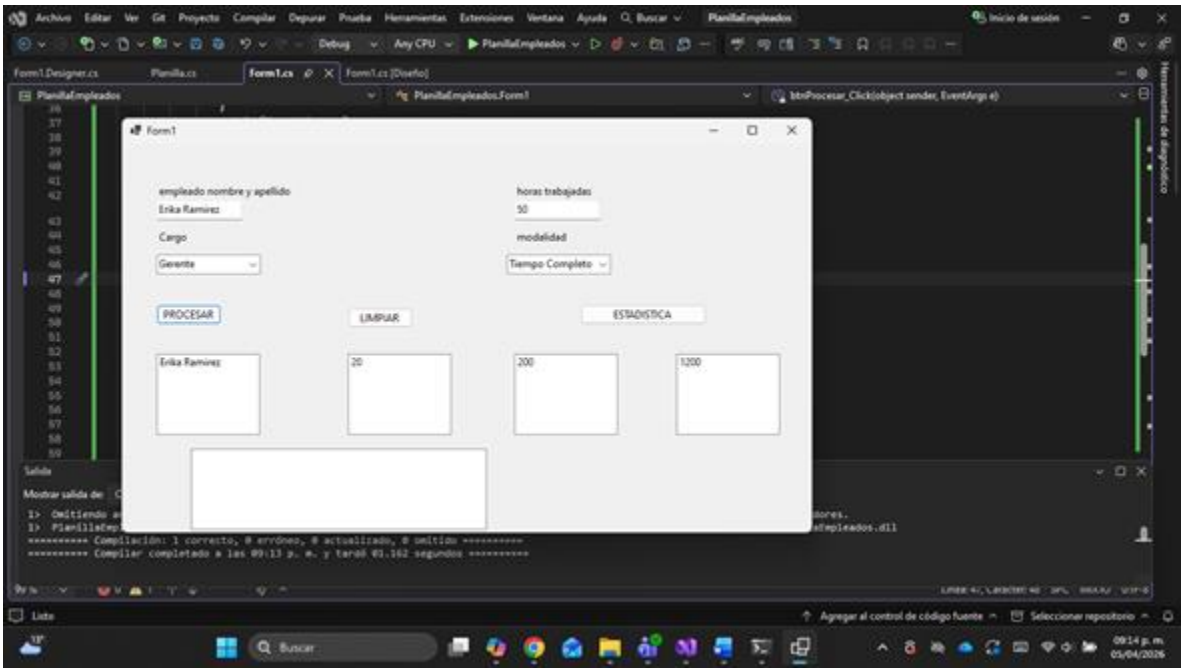
Aun no salen, porque no hay un botón que llame a la base de datos para mostrar los registros añadidos

```
    -> );
Query OK, 0 rows affected (14.626 sec)

mysql> USE planilla;
Database changed
mysql> SELECT * FROM empleados;
Empty set (0.931 sec)
```

Significa que hay que añadir el botón PROCESAR



listo



```
    ->     horas INT NOT NULL,
    ->     cargo VARCHAR(50) NOT NULL,
    ->     modalidad VARCHAR(50) NOT NULL,
    ->     pagoHora DECIMAL(10,2),
    ->     bonificacion DECIMAL(10,2),
    ->     sueldoFinal DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (14.626 sec)

mysql> USE planilla;
Database changed
mysql> SELECT * FROM empleados;
Empty set (0.931 sec)

mysql> USE planilla;
Database changed
mysql> USE planilla;
Database changed
mysql> SELECT * FROM empleados;
Empty set (0.021 sec)

mysql> SELECT * FROM empleados;
+----+----------------+-------+-------+------------+----------+---------------+--------------+
| id | nombre         | horas | cargo | modalidad  | pagoHora | bonificacion | sueldoFinal |
+----+----------------+-------+-------+------------+----------+---------------+--------------+
|  1 | Erika Ramirez  |    50 | 0     | 0          |    20.00 |        200.00 |      1200.00 |
+----+----------------+-------+-------+------------+----------+---------------+--------------+
1 row in set (0.025 sec)

mysql>
```

Pero no guarda Cargo y modalidad, tenemos que modificar el tipo de dato en el botón procesar

```
{
    string nombre = txtEmpleado.Text;
    int horas = int.Parse(txtHoras.Text);
    string cargo = cboCargo.SelectedItem.ToString();
    string modalidad = cboModalidad.SelectedItem.ToString();

    Planilla p = new Planilla(horas, cboCargo.SelectedIndex, cboModalidad.SelectedIndex);


    lstEmpleado.Items.Add(nombre);
```

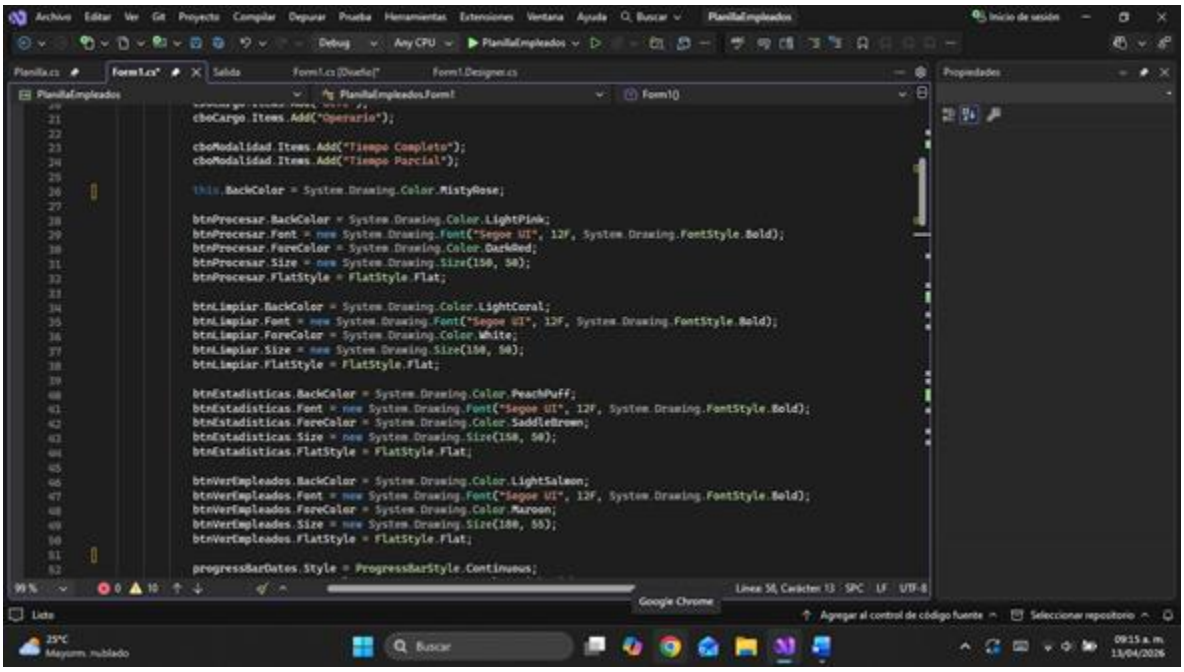
listo

```
mysql> SELECT * FROM empleados;
+----+----------------+-------+-------+------------------+----------+---------------+--------------+
| id | nombre         | horas | cargo | modalidad        | pagoHora | bonificacion | sueldoFinal |
+----+----------------+-------+-------+------------------+----------+---------------+--------------+
|  1 | Erika Ramirez  |    50 | 0     | 0                |    20.00 |        200.00 |      1200.00 |
|  2 | Olivia Benson  |    45 | Jefe  | Tiempo Completo  |     8.00 |         72.00 |       432.00 |
+----+----------------+-------+-------+------------------+----------+---------------+--------------+
2 rows in set (0.012 sec)

mysql>
```

Ahora si guarda esos datos

```
MySQL 9.6 Command Line Cli

Empty set (0.021 sec)

mysql> SELECT * FROM empleados;
+----+----------------+-------+--------+-----------+----------+---------------+-------------+
| id | nombre         | horas | cargo  | modalidad | pagoHora | bonificacion  | sueldoFinal |
+----+----------------+-------+--------+-----------+----------+---------------+-------------+
|  1 | Erika Ramirez  |    50 | 0      | 0         |    20.00 |        200.00 |     1200.00 |
+----+----------------+-------+--------+-----------+----------+---------------+-------------+
1 row in set (0.025 sec)

mysql> SELECT * FROM empleados;
+----+----------------+-------+--------+-----------------+----------+---------------+-------------+
| id | nombre         | horas | cargo  | modalidad       | pagoHora | bonificacion  | sueldoFinal |
+----+----------------+-------+--------+-----------------+----------+---------------+-------------+
|  1 | Erika Ramirez  |    50 | 0      | 0               |    20.00 |        200.00 |     1200.00 |
|  2 | Olivia Benson  |    45 | Jefe   | Tiempo Completo |     8.00 |         72.00 |      432.00 |
+----+----------------+-------+--------+-----------------+----------+---------------+-------------+
2 rows in set (0.012 sec)

mysql> SELECT * FROM empleados;
+----+----------------------+-------+-----------------+-----------------+----------+---------------+-------------+
| id | nombre               | horas | cargo           | modalidad       | pagoHora | bonificacion  | sueldoFinal |
+----+----------------------+-------+-----------------+-----------------+----------+---------------+-------------+
|  1 | Erika Ramirez        |    50 | 0               | 0               |    20.00 |        200.00 |     1200.00 |
|  2 | Olivia Benson        |    45 | Jefe            | Tiempo Completo |     8.00 |         72.00 |      432.00 |
|  3 | Jahaziel Valderrama  |    50 | Administrativo  | Tiempo Completo |    10.00 |        100.00 |      600.00 |
+----+----------------------+-------+-----------------+-----------------+----------+---------------+-------------+
3 rows in set (0.276 sec)
```

Agregamos diseño a la interfaz desde el forms cs



Aquí añadimos la progress bar para que muestre un avance de 25% cada que llenemos un campo del programa

```
progressBarDatos.Style = ProgressBarStyle.Continuous;
progressBarDatos.ForeColor = System.Drawing.Color.LightPink;
progressBarDatos.BackColor = System.Drawing.Color.WhiteSmoke;
progressBarDatos.Maximum = 100;
progressBarDatos.Minimum = 0;
progressBarDatos.Value = 0;

// Conectar eventos para actualizar progreso
txtEmpleado.TextChanged += (s, e) => ActualizarProgreso();
txtHoras.TextChanged += (s, e) => ActualizarProgreso();
cboCargo.SelectedIndexChanged += (s, e) => ActualizarProgreso();
cboModalidad.SelectedIndexChanged += (s, e) => ActualizarProgreso();
}

1 referencia
```

Aquí se va actualizando la progress bar

```
// Metodo para actualizar la ProgressBar
1 referencia
private void ActualizarProgreso()
{
    int progreso = 0;

    if (!string.IsNullOrWhiteSpace(txtEmpleado.Text))
        progreso += 25;

    if (!string.IsNullOrWhiteSpace(txtHoras.Text))
        progreso += 25;

    if (cboCargo.SelectedIndex != -1)
        progreso += 25;

    if (cboModalidad.SelectedIndex != -1)
        progreso += 25;

    progressBarDatos.Value = progreso;
}
}
}
```

Esto muesta la conexión exitosa a la bd

```
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    using (MySqlConnection conn = new MySqlConnection(connectionString))
    {
        try
        {
            conn.Open();
            MessageBox.Show("Conexión exitosa a MySQL");
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error al conectar: " + ex.Message);
        }
    }
}
```

Asi va quedando el diseño

A medida que vamos llenando los datos se va viendo el progreso

Form1
Empleado nombre y apellido
Pablo Gonzalez
Horas trabajadas
50
Cargo
Modalidad
PROCESAR
LIMPIAR
ESTADISTICA
Erika Ramirez
Olivia Benson
Jahaziel Valderrama
20.00
8.00
10.00
200.00
72.00
100.00
1200.00
432.00
Ver empleados
Observa tu progreso